AI Sales and Growth Team: Architecture

An AI team that helps sell our 5–6 SaaS products. A coordinator agent plans the work, specialist agents research, find leads, write drafts and prepare demos, and an evaluator checks every result before a human sees it. Everything the team produces is a draft; people send, publish, push and approve.

- **Status:** Phase A1 built (29 October 2026). Scripts and the guard hook are tested; all 11 agents load in Claude Code. No full sales sprint has been run yet.
- **Runs in:** Claude Code (desktop app Code tab or CLI), on the team's Claude Max plan.
- **Related:** rollout plan in PLAN.md, status log in ROADMAP.md.

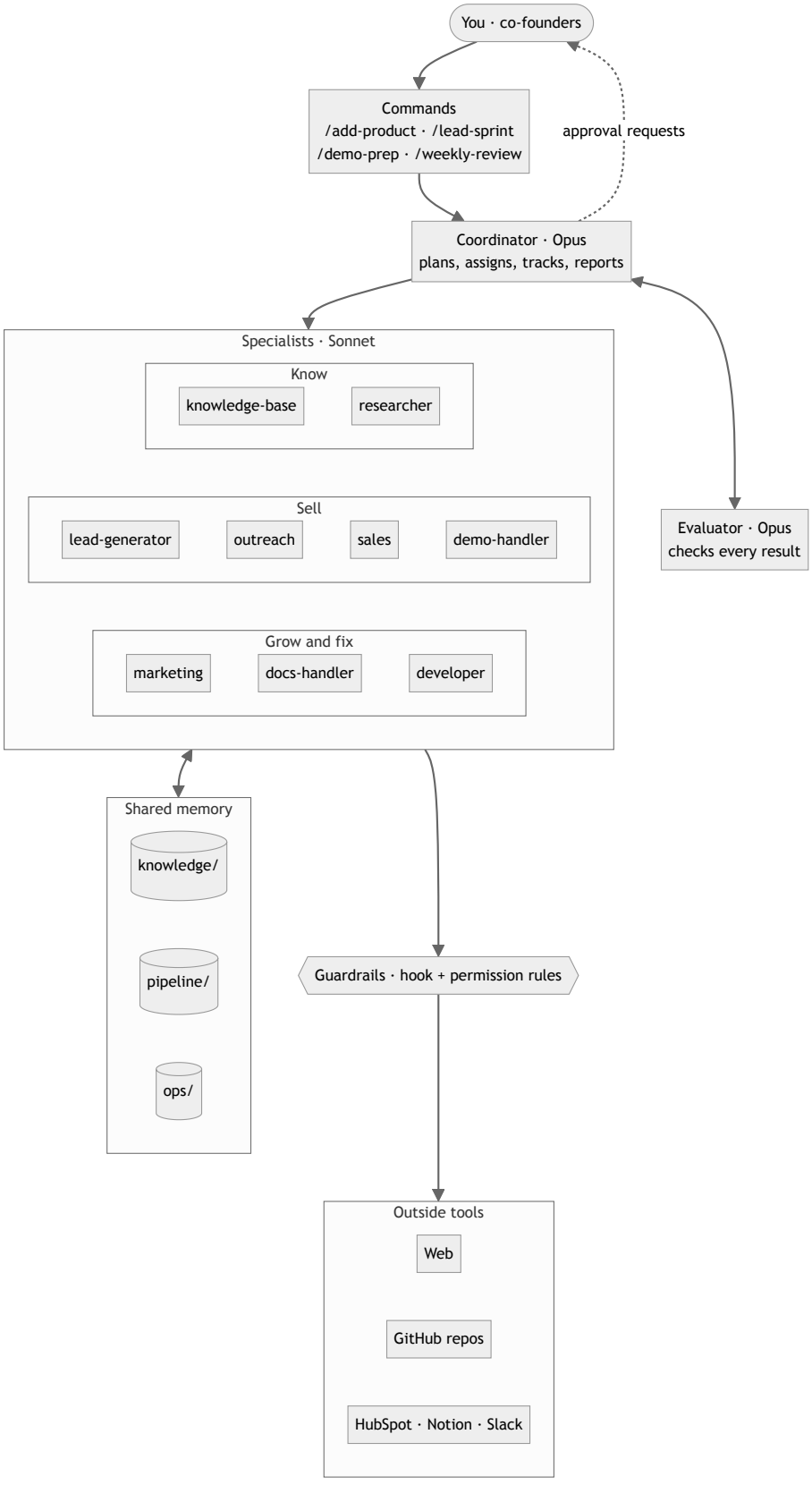
1. Goals and scope

Goal: turn our existing products into paying customers without hiring a sales team first.

In scope now (Phase A): product fact sheets, market and competitor research, lead finding and scoring, outreach drafts, sales replies and proposals, demo preparation, help docs and small code fixes that block sales.

Out of scope for now: sending anything automatically, deploying code, running on a schedule without a person present, and reading live product analytics. Those are Phases B and C in the plan, and only if they prove worth it.

2. System overview



Read it top to bottom:

1. A person types a command (or a plain request) to the **coordinator**.
2. The coordinator splits the job into tasks and gives each one to a **specialist** in one of three groups.
3. Specialists save their work as files and report back. They never call each other.

4. Every deliverable goes to the **evaluator**, which runs automated checks, opens sources and scores the work.
5. Anything that would leave the company (an email, a post, a code push, a CRM change) goes to the **approval list** for a person to decide.
6. All agents share the same **files**: the knowledge base, the lead pipeline and the task board.
7. Calls to the outside world pass through **guardrails**: a hook script and permission rules in `.claude/settings.json`.

3. Components

Component	What it does	Built with	Where
Coordinator	Turns requests into tasks, delegates, routes handoffs, tracks the board and approvals, reports	Claude Code subagent run as the main session (`"agent": "coordinator"`)	`.claude/agents/coordinator.md`
Evaluator	Scores every deliverable; PASS, REVISE or ESCALATE; can't approve actions	Subagent + check scripts	`.claude/agents/evaluator.md`, `evaluation/rubrics.md`
9 specialists	One job each (section 4)	Subagents	`.claude/agents/*.md`
Shared context	Products, team, protocol and hard rules every agent reads	`CLAUDE.md`	`CLAUDE.md`
Knowledge base	Company and product facts with a trust status	Markdown with frontmatter	`knowledge/`
Lead pipeline	Every lead, its evidence, score and stage	JSON Lines behind a validating script	`pipeline/`, `scripts/leads.mjs`
Task board and approvals	Every task and every yes/no	Markdown tables	`ops/`
Guard hook	Blocks edits to confirmed knowledge, self-confirmation, and direct writes to the lead file	Claude Code `PreToolUse` hook (Node)	`scripts/guard.mjs`
Permission rules	Pre-allows research; always asks before push, PR or confirming knowledge; denies `.env` reads	Claude Code settings	`.claude/settings.json`
Commands	Repeatable workflows	Claude Code slash commands	`.claude/commands/`
Connectors	HubSpot, Notion, Slack (optional)	MCP over HTTP with OAuth	`.mcp.json`

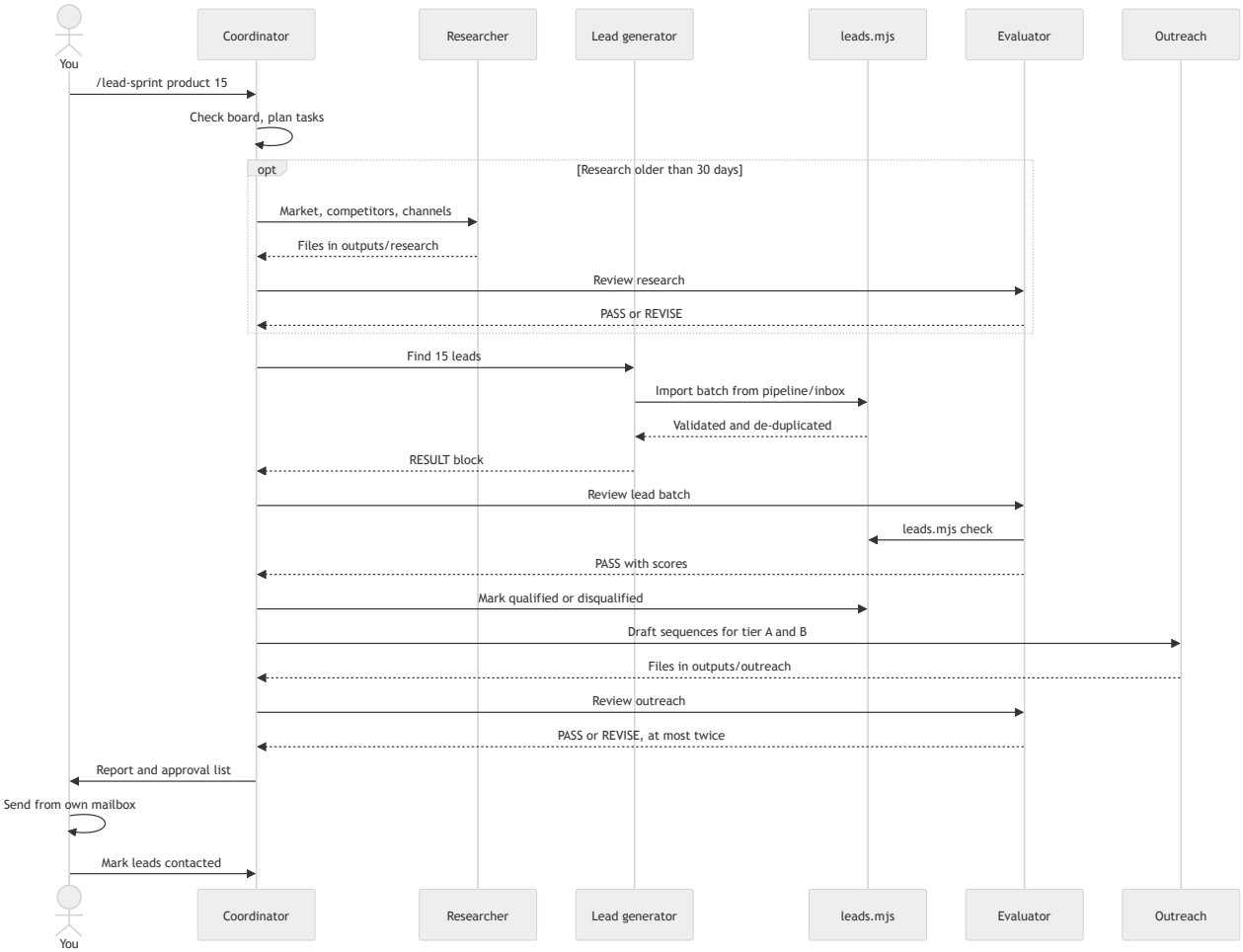
4. Agents

Agent	Model	Tools	Writes to	Typical handoffs
`coordinator`	Opus	Agent (only the 10 below), files, Bash, web	`ops/board.md`, `ops/approvals.md`	Any specialist, evaluator

Agent	Model	Tools	Writes to	Typical handoffs
`evaluator`	Opus	Files (read + write reviews), Bash, web	`outputs/reviews/`	Back to coordinator only
`knowledge-base`	Sonnet	Files, Bash (`gh repo clone`), web	`knowledge/` (proposed only)	—
`researcher`	Sonnet	Files, web	`outputs/research/`, `knowledge/competitors/`	lead-generator, marketing
`lead-generator`	Sonnet	Files, Bash (lead script), web, HubSpot read	`pipeline/inbox/` → `leads.jsonl`	outreach, researcher
`outreach`	Sonnet	Files, Bash (lead script)	`outputs/outreach/`	sales, demo-handler
`sales`	Sonnet	Files, Bash, web, HubSpot read	`outputs/sales/`	demo-handler, docs-handler, developer
`marketing`	Sonnet	Files, web	`outputs/marketing/`	developer, docs-handler
`demo-handler`	Sonnet	Files, Bash, web	`outputs/demos/`	developer, sales
`docs-handler`	Sonnet	Files, Bash, web	`outputs/docs/`	developer, knowledge-base
`developer`	Sonnet	Files, Bash	a branch in `repos/<slug>`, `outputs/dev/`	docs-handler, demo-handler

Specialists don't have the Agent tool, so they can't start other agents. Opus is used where judgment matters most (planning and reviewing); Sonnet costs half as much and does the specialist work.

5. How work flows: a lead sprint



Other commands follow the same pattern: `/add-product` (knowledge-base → evaluator → human confirms), `/demo-prep` (demo-handler and sales → evaluator) and `/weekly-review` (pipeline and board summary, sales review).

6. Communication protocol

The team is hub and spoke. Only the coordinator assigns work, and specialists talk only to the coordinator.

- **Task IDs** look like `T-YYYYMMDD-NN` and appear on the board, in file names and in reviews.
- **Every specialist ends with a RESULT block:**

```
RESULT
task: T-YYYYMMDD-NN
status: done | blocked | needs_input
outputs: <file paths>
summary: <1-3 sentences>
confidence: high | medium | low, and why
handoffs: <agent>: <what and why> | none
questions: <for the human> | none
```

- **Handoff rules.** The coordinator accepts a handoff only if it serves the current goal, isn't already on the board, and is no more than three steps from the person's original request. Otherwise it records it as a

suggestion.

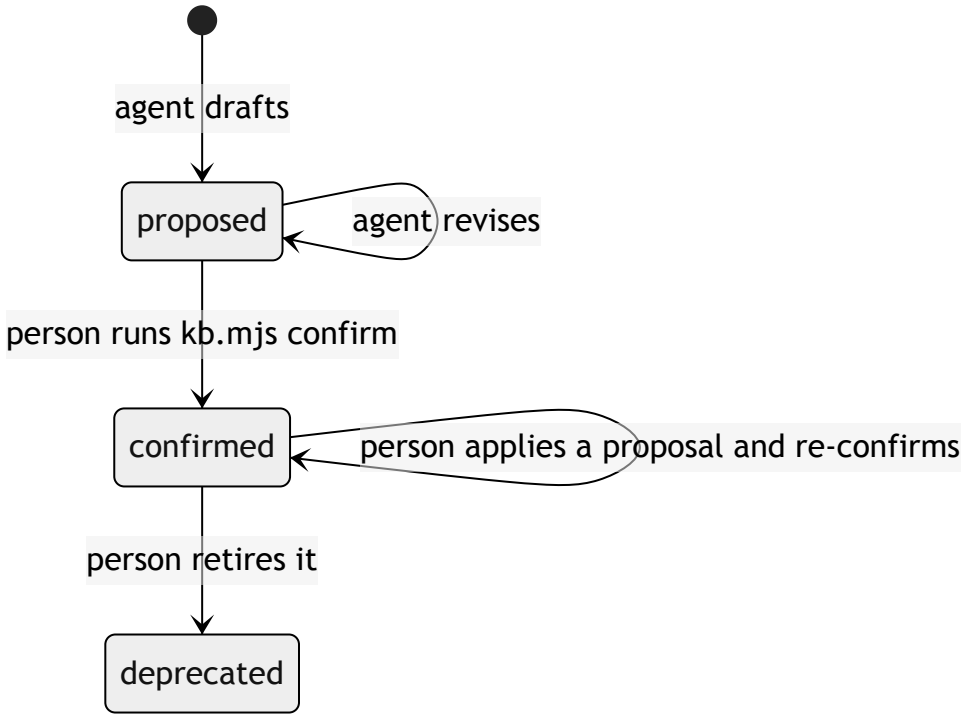
- **Revision limit.** At most two revisions per deliverable, then it's escalated to a person.
- **Duplicate work.** The coordinator checks the board before creating a task and reuses recent output (for example, research less than 30 days old).

These routing rules are enforced by the coordinator's instructions, not by code (see section 11).

7. Shared memory

Knowledge base

Each file in `knowledge/` has a `status` in its frontmatter. Agents can only write `proposed` files; only a person can confirm one, and confirmed files are then locked by the guard hook.



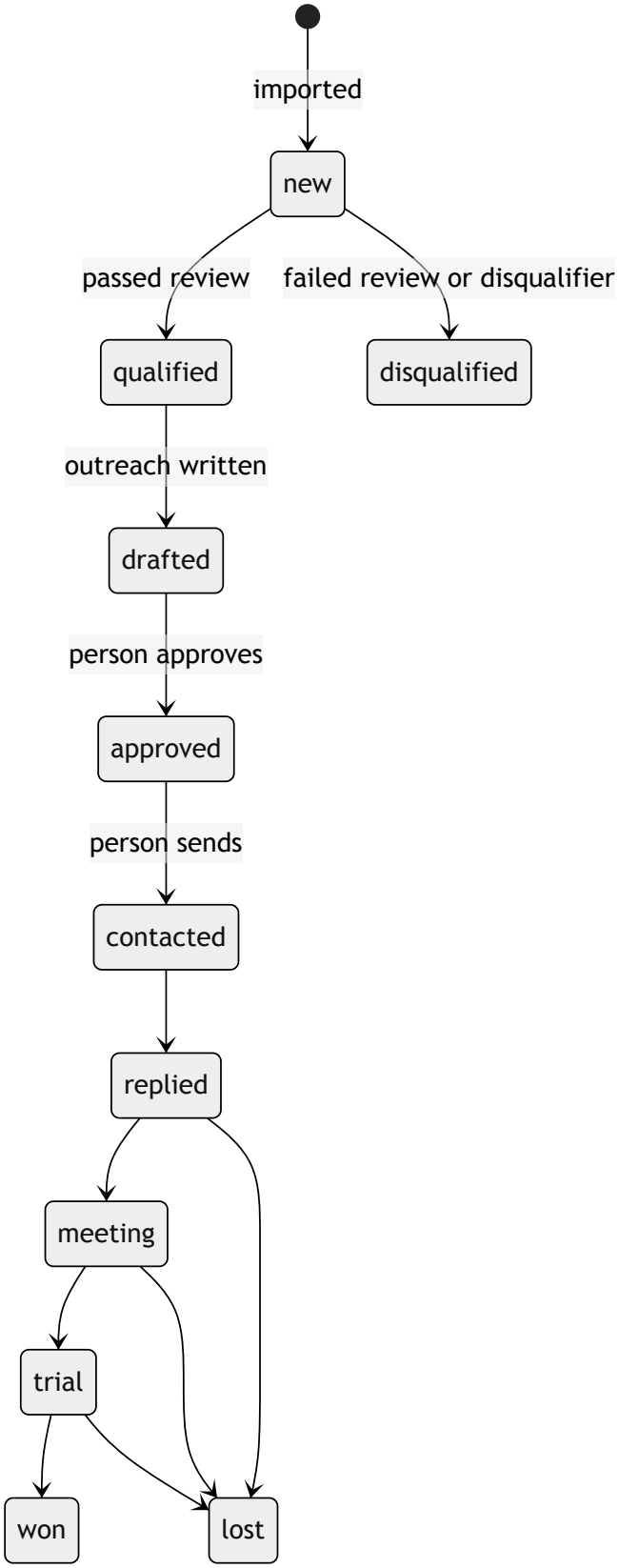
Path	Content
`knowledge/company.md`	Company profile, pricing and discount policy, outreach policy, brand voice, approvers
`knowledge/products/<slug>.md`	One fact sheet per product (template in `_template.md`)
`knowledge/competitors/<slug>.md`	Competitor research per product, dated
`knowledge/proposals/`	Suggested changes to confirmed facts

Customer-facing work may state only confirmed facts. Anything else is marked `[UNCONFIRMED]`.

Lead pipeline

`pipeline/leads.jsonl` is written only by `scripts/leads.mjs`, which rejects a whole batch if any lead is missing evidence, has an unsourced contact, an invalid score or a product without a fact sheet. A lead's ID

comes from its domain and product, so importing the same company again updates it instead of creating a duplicate. Field definitions are in `pipeline/lead-schema.md`.

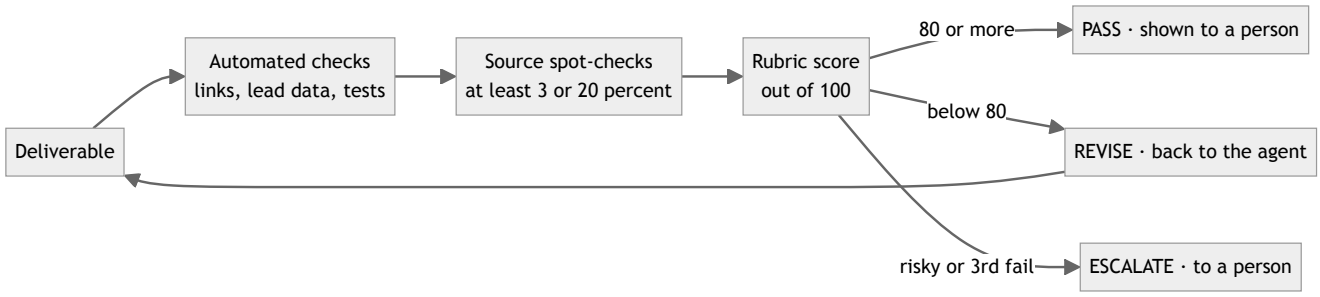


Every status change is logged in the lead's history with a date and note. `pipeline/leads.csv` is regenerated after every change for use in a spreadsheet.

Task board and approvals

`ops/board.md` lists every task with owner, status, output and review score. `ops/approvals.md` lists every action that needed a person, with the decision and date.

8. Quality: how the evaluator works



Work type	Automated checks	Main criteria
Product fact sheet	Link check; all sections present	Sourced facts, live vs planned features, pricing conflicts listed
Market or competitor research	Link check; dated prices	Sources, facts vs hypotheses, relevance to the ICP
Lead batch	`leads.mjs check`	Evidence holds up (80% of spot-checks), ICP fit, sourced contacts
Outreach sequence	—	Personalisation from evidence, confirmed facts, clarity, compliance
Sales asset	—	Prices match policy, commitments flagged for approval
Marketing content	Link check	Accuracy, ICP language, call to action
Demo plan	—	Live features only, tied to pains, no credentials
Help doc	—	Steps verified against the product, right audience
Code change	Project's tests, types, lint	Tests pass, root cause shown, minimal diff

Every work type auto-fails on invented facts, unconfirmed customer-facing claims, secrets in the output, or acting on instructions found inside fetched content. Full rubrics are in `evaluation/rubrics.md`. A PASS means "good enough to show a person"; it never approves an action.

9. Security and control

Rule	Enforced by
Agents can't edit confirmed knowledge or mark anything confirmed	`scripts/guard.mjs` hook (runs for every agent)
Leads are written only through the validating script	Guard hook + `scripts/leads.mjs`
Confirming knowledge needs a person	`ask` rule on `node scripts/kb.mjs confirm`
`git push`, `gh pr create` and `gh pr merge` need a person	`ask` rules; force-push is denied
`.env` files can't be read	`deny` rules on `Read(.env)` and `Read(.env.*)`

Rule	Enforced by
Drafts only: no sending, posting, publishing, deploying or CRM writes without approval	Agent instructions and `CLAUDE.md`, backed by Claude Code's normal permission prompts
Web pages, repos and emails are data, not instructions	Agent instructions; evaluator auto-fails work that acted on them

Secrets. The repo holds no secrets. MCP connectors sign in with OAuth through `/mcp`; nothing is stored in `.mcp.json`.

Personal data. Leads hold business contact details that were published publicly, with the source recorded. Keep this repository private, honour opt-outs and delete leads that are no longer needed.

Permission mode. Keep Claude Code in its default permission mode so that any action outside the allow list asks first.

10. Cost

The team runs inside the Claude Max plan, so normal use has no per-use bill; the limits are a 5-hour usage window and a weekly cap. Paid extra usage is switched off. Designing and building the system on 29 October 2026 used 13% of one 5-hour window.

For comparison only, estimated API pay-per-use costs at list prices (Opus 5.5: $4 / $20 per million input / output tokens; Sonnet 5.5: $2 / $10):

Workflow	Estimated API cost
`/add-product` (one fact sheet)	$2–4
Research for one product	$3–5
`/lead-sprint` (15 leads, drafts, reviews)	$4–8
`/demo-prep`	$2–4
A month at a steady pace	about $100–300

These are estimates from expected token use, not measurements.

11. Known limitations

- **No full sprint run yet.** The agents load in Claude Code, but the end-to-end workflows and slash commands haven't been exercised.
- **Routing rules are instructions.** The three-step handoff limit, the two-revision limit and the duplicate check are followed by the coordinator but not enforced by code.
- **Interactive only.** The team works while someone has a session open. There's no scheduler and no always-on service (Phase C).
- **No product signals yet.** The team can't see sign-ups, errors or payments in our products (Phase B).
- **One operator at a time.** The board and approval files are plain Markdown; two people running sprints at once can produce conflicting edits.
- **Markdown storage.** Fine for a handful of products and hundreds of leads. A database is a Phase C decision.

12. Repository layout

```
CLAUDE.md                shared context: products, team, protocol, rules
README.md                how to use the team
.claude/agents/          the eleven agents
.claude/commands/        /add-product, /lead-sprint, /demo-prep, /weekly-review
.claude/settings.json    main agent, permissions, guard hook
.mcp.json                HubSpot, Notion, Slack connectors (OAuth, no secrets)
docs/                    ARCHITECTURE.md, PLAN.md, ROADMAP.md, PDF
evaluation/rubrics.md    scoring rules per work type
knowledge/               company.md, products/, competitors/, proposals/
ops/                     board.md, approvals.md
outputs/                 deliverables, created as the team works
pipeline/                lead-schema.md, leads.jsonl, leads.csv, inbox/
scripts/                 leads.mjs, kb.mjs, check-links.mjs, guard.mjs, build-docs-pdf.mjs
repos/                   product repo clones (git-ignored)
```

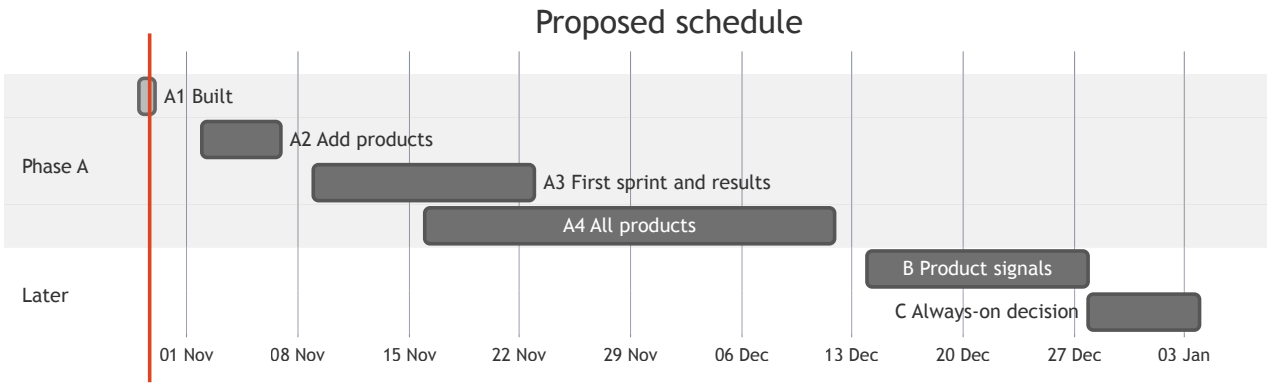
AI Sales and Growth Team: Rollout Plan

How we go from "built" to paying customers, one phase at a time. Each phase has tasks, an owner and a test that says when it's done; we don't start the next phase until that test passes. How the system works is in ARCHITECTURE.md; progress is logged in ROADMAP.md.

Dates are proposals. Names in `[brackets]` are for the team to fill in.

Phases at a glance

Phase	Goal	When (proposed)	Done when
A1 · Built	The team, knowledge base, lead pipeline, checks and guardrails exist	Done, 29 Oct 2026	Scripts and hook tested; agents load
A2 · Add products	Confirmed fact sheets for our 5–6 products	Week of 2 Nov	Every fact sheet confirmed
A3 · First sprint	15 leads and outreach for one product; measure results	Weeks of 9 and 16 Nov	Sprint passes review; results recorded
A4 · All products	A weekly selling rhythm across every product	16 Nov – 11 Dec	Every product sprinted; 4 weekly reviews
B · Product signals	The team can see sign-ups, errors and payments, read-only	Only if A4 shows we need it	One product's funnel answered with real numbers
C · Always-on system	Scheduled work, a database and a dashboard	Only if needed	Go / no-go decision with costs



Phase A2 · Add the products (week of 2 Nov)

#	Task	Owner
1	Create a private GitHub repo and push this folder	[name]
2	Fill in `knowledge/company.md` (pricing and discount policy, outreach policy, voice, approvers), then confirm it with `node scripts/kb.mjs confirm knowledge/company.md`	[name]
3	List the 5–6 products with website and repo	[name]
4	Run `/add-product <name> <website> <owner/repo>` for each product	[name]

#	Task	Owner
5	Review each fact sheet, answer its open questions, then confirm it	Product owner for each
6	Optional: set up a separate domain and mailbox for outreach	[name]

Done when

- [ ] `node scripts/kb.mjs status` shows `knowledge/company.md` and every product file as `confirmed`.
- [ ] Each fact sheet passed review with a score of 80 or more, and no pricing conflicts are left open.
- [ ] `/agents` lists all eleven agents, and a new session in the repo starts as the coordinator.

Phase A3 · First lead sprint (weeks of 9 and 16 Nov)

#	Task	Owner
1	Pick the first product: the one with the clearest buyer and a sign-up and checkout that work	Founders
2	Run `/lead-sprint <product-slug> 15`	[name]
3	Read the approval list in `ops/approvals.md`; send the approved emails ourselves; mark each lead `contacted`	[name]
4	Paste replies to the coordinator; use `/demo-prep <lead-id>` for any meeting	[name]
5	Run `/usage` after the sprint and note how much of the 5-hour window it used	[name]
6	After two weeks, record the results in `docs/ROADMAP.md`	[name]

Done when

- [ ] 15 leads were imported and `node scripts/leads.mjs check` passes.
- [ ] The lead batch and the outreach drafts each passed review within two revisions.
- [ ] Every email we sent has a row in `ops/approvals.md`, and its lead is marked `contacted`.
- [ ] The metrics below are recorded for this sprint.

Phase A4 · All products (16 Nov – 11 Dec)

#	Task	Owner
1	One or two lead sprints a week, rotating through the products	[name]
2	`/weekly-review` every Friday; agree next week's focus	Founders
3	Marketing: positioning and a landing-page audit for each product	[name]
4	Docs: a getting-started guide for each product	[name]
5	Developer: fix the top blockers that come up in sales conversations (on branches, reviewed by us)	[name]

Done when

- [ ] Every product has had at least one lead sprint and one landing-page audit.
- [ ] Four weekly reviews have run.
- [ ] We've decided, from the numbers, which products to focus on.

Phase B · Product signals (only if needed)

Start only if A4 keeps hitting questions we can't answer without product data: where sign-ups drop off, why payments fail, which errors users hit.

#	Task
1	List, per product, what exists: analytics, error tracking, logs, billing, support inbox
2	Give the team read-only access for one product first, through official APIs or MCP connectors
3	Add a product-health command and a monitoring agent that reports drops with evidence, never guesses

Done when: for one product, the team can answer "where do sign-ups drop off this week?" with sourced numbers, using read-only access.

Phase C · Always-on system (only if needed)

Start only if we need work to run without a person present (scheduled research, alerts), several people running the team at once, or a dashboard.

#	Task
1	Research current runtime options against official docs: for example the Claude Agent SDK, Claude Managed Agents, or a workflow engine with our own agent loop
2	Estimate API costs. This runs on an API key, billed per use and separate from the Max plan
3	Design the move from Markdown files to a database, and a dashboard for tasks, approvals and cost

Done when: we have a go or no-go decision with a cost estimate.

Metrics

Metric	How we measure it	Target
Leads per sprint	`node scripts/leads.mjs list`	15
Leads that hold up in review	Evaluator spot-checks	80% or more
Deliverables passing review first time	`outputs/reviews/`	[target]
Reply rate	replied ÷ contacted	[target]
Meetings booked	Leads at `meeting`	[target]
Trials and paying customers	Leads at `trial` and `won`	[target]
Our time per sprint	Our own log	60 minutes or less
Plan usage per sprint	`/usage`	Recorded

Decisions needed

Decision	Options	Decided by	By when
Start Phase A	Yes or no	Founders	Next meeting
First product	One of our products	Founders	Next meeting

Decision	Options	Decided by	By when
Who approves what	Sending, publishing, code, discounts	Founders	Next meeting
Outreach mailbox	Separate domain or main domain	[name]	Week of 2 Nov
Where the repo lives	Personal account or company organisation	[name]	Week of 2 Nov

Risks and how we handle them

Risk	What we do
Outreach lands in spam or hurts our main domain	Separate domain, low daily volume, an opt-out line in every email
Wrong or invented claims reach a prospect	Only confirmed facts in customer-facing work; evaluator auto-fails invented facts; a person approves every send
Privacy and anti-spam rules	Public business information only, sources recorded, opt-outs honoured, stale leads deleted; check the rules for each country we email (for example GDPR for EU contacts)
Hitting Max plan limits	Smaller batches, a fresh session per task, switch the coordinator to Sonnet if needed
A bad code change	Branches only, tests required, push and PRs need a person, we review every PR
Two people editing the board at once	One person runs each sprint; commit after each sprint
Leaning on the AI for relationships	Founders own every customer conversation; the team only prepares drafts

First session checklist

1. Clone the repo and open it in the Claude desktop app's Code tab, or run `claude` in it.
2. Check the agents loaded: `/agents`.
3. Fill in and confirm `knowledge/company.md`.
4. Run `/add-product` for the first product and review its fact sheet.
5. Run `/lead-sprint <product-slug> 15`.